1. Objectives:

1. To understand the concept of polymorphism in Java programming.
2. To learn the use of abstract classes and abstract methods in OOP.
3. To understand method overriding and runtime polymorphism.

2. Introduction:

- **Polymorphism:** It means "many forms," where the same method or object can behave differently depending on the situation. In Java, polymorphism is commonly achieved through method overriding, where a child class provides its own implementation of a method inherited from the parent class. This helps increase flexibility, reusability, and maintainability of programs.

- **Abstract Class:** It is a special type of class that cannot be instantiated directly. It is declared using the abstract keyword and is mainly used as a base class for other classes. An abstract class may contain both abstract methods and normal methods. An **abstract method** is a method without a body, and it must be implemented by the child classes that inherit the abstract class. This ensures that different child classes provide their own specific implementation of the method.

3. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

4. Experiment Implementation:

Experiment 1: Use of Static keyword

```
package Polymorphism;
public class StudentInfo {

    public void display() {
        System.out.println("I'm a student");
    }
}

package Polymorphism;
public class StudentOne extends StudentInfo {

    public void display() {
        System.out.println("I'm student one");
    }
}


package Polymorphism;
public class StudentTwo extends StudentInfo {

    public void display() {
        System.out.println("I'm student two");
    }
}

package Polymorphism;
public class Main {
    public static void main(String[] args) {

        StudentInfo s1 = new StudentInfo();
        s1.display();

        s1 = new StudentOne();
        s1.display();

        s1 = new StudentTwo();
        s1.display();
    }
}
```

Experiment 1 output:

```
PS D:\BAUST CSE\Java Programing\8th_Lab\1st Code>  & 'C:\Program Files\Java\jdk-25.0.2\bin\ja
:\BAUST CSE\Java Programing\8th_Lab\1st Code\bin' 'Polymorphism.Main'
I'm a student
I'm student one
I'm student two
PS D:\BAUST CSE\Java Programing\8th_Lab\1st Code>
```

Experiment 2: Use of this keyword

```
package Polymorphism;

public abstract class Shape {

    abstract double area();
}

package Polymorphism;
public class Rectangle extends Shape {
    double length, width;

    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    @Override
    double area() {
        System.out.println("Rectangle area: " + (length * width));
        return 0;
    }
}

package Polymorphism;
public class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }
    @Override
    double area() {
        System.out.println("Triangle area: " + (0.5 * base * height));
        return 0;
    }
}

package Polymorphism;
public class Circle extends Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }
    @Override
    double area() {
        System.out.println("Circle area is: " + (3.1416 * radius * radius));
        return 0;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s;

        s = new Rectangle(10, 5);
        s.area();
        s = new Triangle(10, 5);
        s.area();
        s = new Circle(10);
        s.area();
    }
}
```

Experiment 2 output:

```
PS D:\BAUST CSE\Java Programing\8th_Lab>  & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe
ezwan\AppData\Roaming\Code\User\workspaceStorage\21859bc624e912665c55dde0977718be\redhat.j
Rectangle area: 50.0
Triangle area: 25.0
Circle area is: 314.16
PS D:\BAUST CSE\Java Programing\8th_Lab>
```

5. Discussion & Conclusion:

In this lab, the concepts of polymorphism and abstract classes were implemented using Java programs. Method overriding was used to achieve runtime polymorphism, where the same method behaved differently in different child classes. The abstract class was used as a base class to define common structures and abstract methods, which were later implemented in derived classes. We understood abstraction, inheritance, and polymorphism. Overall, this lab improved the knowledge of advanced object-oriented programming concepts.

7. Reference:

1. https://www.geeksforgeeks.org/java/polymorphism-in-java/ accessed at 5.20 PM on 16th May 2026.
2. https://www.geeksforgeeks.org/java/abstract-classes-in-java/ accessed at 5.45 PM on 16th May 2026.